Pairing Proofs and Polynomial Ring Toolkit

Revision requirements and responses — Major Revision resubmission

Resubmission: TCHES 2027, Issue 1, Paper #185
Original submission: TCHES 2026, Issue 4, Paper #104 (decision: Major Revision)
Accompanying document: the original submission PDF, provided alongside this document as required.

This document lists the revision requirements for our major revision and explains how each was addressed. **Part A** covers the binding revision criteria collected by the reviewers after the post-rebuttal discussion (Comment @A1 on submission #104). **Part B** covers the remaining required changes, questions, and editorial comments from the individual reviews (104A, 104B, 104C). **Part C** summarises the location of all changes in the revised paper. Section, table, and algorithm numbers refer to the *revised* paper unless marked "orig."

Part A — Binding revision criteria (Comment @A1)

> **A.1** *"Clarify the main contribution relative to the most closely related prior works (such as Fel23 and Garaga)"*

- **Section 3 (Related Work) was rewritten** as an enumerated treatment of the three works we build on — [Hou23] (pairings in R1CS), [Fel23] (polynomial identity testing for $F_{q^{12}}$ arithmetic), and [NE24] (residue-witness final exponentiation) — with each item now ending in a precise statement of what this paper changes relative to it.
- **Versus [Fel23]**, the delta is now stated quantitatively: we apply the same polynomial framework at a coarser granularity — entire Miller loop steps as single polynomial relations instead of individual extension-field multiplications — eliminating intermediate remainder commitments. Concretely, a 3-pair zero-bit Miller loop step requires four chained relations and 48 F_q commitments under [Fel23], versus one relation and 12 commitments in our scheme (Section 3, item 2; derivation in Section 4.1).
- **Versus Garaga**, a dedicated paragraph now closes Section 3: Garaga deployed an early, Cairo-specific proof-of-concept of the consolidation technique in 2024, adapted from a preceding technical write-up (cited anonymously), and provides no formal treatment. This paper supplies the missing foundations — the operation polynomials of Section 4 and the batched two-round protocol with its soundness analysis in Section 5 — and evaluates the technique systematically against the strongest available baseline (gnark's emulated pairing stack, incorporating [Hou23] and [NE24]). Section 7.2 additionally disambiguates the two implementations and their respective metrics.
- The contribution list in Section 1.2 was rewritten to match.

> **A.2** *"Add additional benchmarks for BLS12-381 for a more comprehensive evaluation, as per the rebuttal"*

- The gnark toolkit is now implemented for **BLS12-381** (`sw_bls12381`) in addition to BN254, and **Table 7 has been extended with full BLS12-381 results** for both SCS (Plonkish) and R1CS constraint systems: 33.2% / 32.5% constraint reduction, 54.5% reduction in committed elements, and up to 48.8% reduction in solver memory — confirming that the improvements carry over to a modern 128-bit-security curve.
- The CairoVM BLS12-381 measurements are retained in Table 6, so both environments now report both curves.
- The abstract and Section 1.2 quote headline numbers for both curves.

> **A.3** *"Add clarification about the benchmarking setup, as per the rebuttal, in particular for the 3-pairing setting"*

- We simplified the benchmark itself so that the compared circuits are identical: a new **"Benchmark circuit"** paragraph in Section 7.2 specifies that both the baseline and our implementation are evaluated on the same 3-pairing multi-pairing check $\prod_{i=1..3} e(P_i, Q_i) = 1$ via gnark's `PairingCheck`, and explains why this corresponds to the pairing workload of a Groth16 verification (the constant $e(\alpha, \beta)$ term from the trusted setup folds into a product-of-pairings over three variable pairs). This replaces the original submission's composite "Groth16 simulation" (2-pair fixed-G2 check plus an accumulated single pairing, orig. Section 7.2), which reviewers found unclear; all benchmark numbers were regenerated accordingly (abstract, Section 1.2, Tables 6 and 7).
- The same 3-pairing configuration is used for the CairoVM measurements in Table 6, enabling a like-for-like comparison across environments.
- The baseline's provenance is stated explicitly: the upstream gnark `emulated` package, which represents the state of the art for emulated pairings and includes the optimizations of [Hou23] and [NE24] (Section 7.2). It is also stated that **both** implementations exploit the sparseness of the Miller line functions (Section 7.2, with the analysis in Section 4.1).
- The benchmark circuit and our implementation fork are linked from Section 7.2 for reproducibility. On generality beyond the 3-pairing setting, Section 4 gives per-step constraint counts, and the degree of the step polynomial scales as $\deg(P) \approx 88 + 18(k-3)$ for k-pair products (footnote in Section 4), from which costs for other pair counts follow.

> **A.4** *"Improve the mapping between algorithms and implementation, in particular the application of Fiat-Shamir heuristic"*

- **Section 5.4 (Interactive Proof)** now presents the two-round protocol as an explicit prover/verifier message diagram built directly on Algorithms 4 and 5.
- **New Section 5.5 (Fiat-Shamir Transformation)**: the transcript is defined explicitly — $z = H(\text{pairing inputs}, c, W^w, \mathscr{R})$ and $x = H(z, Q_{acc})$ — followed by the resulting non-interactive proof (Section 5.5.1) and an argument for why the soundness of Theorem 1 carries over (every prover-chosen value is absorbed before the challenge it must not depend on). The concrete knowledge-soundness bound $\delta_s + 3(m^2+1)\cdot 2^{-\lambda}$ is stated and instantiated in Section 5.6.
- **New Section 6.3 (Algorithms and Implementation)** maps the protocol onto the code, as promised in the rebuttal: `MulPolyRings` / `polyRingMulHint` perform the prover's work of Algorithm 4 inline during witness generation, queueing each $(A_j; R; Q)$ tuple in the deferred check list;
  `performDeferredRingChecks`, invoked once at circuit finalisation, implements the batched identity of Section 5.1 via two sequential `api.Commit` calls (producing z, then x) followed by one `AssertIsEqual` per ring group — the verifier's work of Algorithm 5. A "Shared Circuit Skeleton" subsection explains how one circuit function per curve traces both algorithms.

> **A.5** *"Qualify the commitment-based performance metric by specifying the various schemes, their overhead and impact on overall performance"*

- **New Section 5.5.2 (Instantiating the Random Oracle)** qualifies the committed-element counts for both instantiations:
  - **CairoVM (Poseidon):** prover hints arrive as transaction calldata and are absorbed into a running Poseidon sponge (two 96-bit limbs per `Felt252` absorption); hashing cost and calldata both scale linearly with the number of committed elements — which is why COMMITS is reported alongside modular operations in Table 6.
  - **gnark (`api.Commit`, per [BSB23] / [BSB]):** costs are backend-specific and are now spelled out. Groth16: two G_1 MSMs per commitment, one extra G_1 proof element per commitment plus a single folded knowledge proof, one extra scalar multiplication and a two-pairing knowledge check for the verifier, and *no* impact on the constraint count. PLONK: one constraint per committed wire, one G_1 element per commitment, and the evaluation folds into the batched KZG opening already performed.
- Metric definitions were tightened: the Table 6 footer defines MULMOD, ADDMOD, COMMITS, and VM STEPS with a cross-reference to Section 5.5.2; the Table 7 metric is renamed "Nb F_q Committed" and footnoted as elements committed via `api.Commit`; commitment terminology was unified across the abstract, introduction, and background.

> **A.6** *"Minor presentation issues pointed out by the reviewers"*

All editorial points were addressed; see the itemised responses in Part B (in particular the 104C editorial comments and 104A's presentation remarks). Float count was reduced by removing two algorithm floats and the duplicated non-interactive protocol figure.

Part B — Individual review points

Review 104A

> *Required: add a modern set of parameters, such as BLS12-381. Question: why BN254?*

BLS12-381 benchmarks added throughout — see A.2. BN254 is retained alongside it because it remains ubiquitous in deployed systems (Ethereum precompiles and the rollup ecosystems built on them), so improvements on it are of direct practical relevance; BLS12-381 provides the modern 128-bit-security data point.

> *Required / question: relationship with Garaga, and novelty over the approach implemented there.*

See A.1: new closing paragraph of Section 3 and clarified Section 7.2. Garaga's 2024 deployment was an early Cairo-specific proof-of-concept adapted from our own circulated write-up (anonymity is therefore not affected); this paper contributes the formal grounding (Sections 4–5), the reusable gnark toolkit (Section 6), and the systematic evaluation against the strongest gnark baseline (Section 7).

> *Question: do you exploit the sparseness of the Miller line functions?*

Yes — both our implementation and the baseline exploit line sparseness; this is now stated explicitly in Section 7.2, and the sparse representation and the resulting constraint savings are accounted for in the Section 4.1 counts and Section 6.1.

> *Question: application scenario for the 3-pairing benchmark; is it the Section 7.2 Groth16 circuit? Give metrics for single and n pairings.*

Yes — the benchmark corresponds to the pairing workload of Groth16 verification, and the revised paper now uses one uniform 3-pairing product check for both implementations, described precisely in the new "Benchmark circuit" paragraph (Section 7.2); see A.3. For other pair counts, per-step constraint counts and the $\deg(P) \approx 88 + 18(k-3)$ scaling in Section 4 allow extrapolation to single and n-pairing settings.

> *Presentation: redundant floats; Algorithms 4 and 5 could be combined; interactive and non-interactive versions are very similar.*

We removed two algorithm floats (polynomial multiply-divide and product evaluation) and the separate non-interactive protocol figure: the non-interactive version is now given textually in Section 5.5.1 as a delta over the interactive protocol. We kept Algorithms 4 and 5 as separate floats because the revision now uses them as the anchor for the implementation mapping (prover work vs. deferred verifier work run at different phases in gnark — Section 6.3), but their shared structure is made explicit in the "Shared Circuit Skeleton" subsection.

Review 104B

> *1. Clarify the main contribution relative to the most closely related prior works.*

See A.1 (Sections 1.2 and 3).

> *2. Improve the presentation of the interactive proof / Fiat-Shamir discussion and its relation to the gnark implementation.*

See A.4 (Sections 5.4, 5.5, 6.3).

> *3. Clean up notation and terminology.*

Commitment terminology was unified ("committed elements" vs. cryptographic commitments, Table 7 metric renamed); Section 5.6 opens with an explicit notation convention separating scalar challenges $z, x \in \mathbb{F}_q$ from the formal indeterminate X; several notational inconsistencies flagged by the reviewers were fixed alongside.

> *4. Clarify the experimental metrics and benchmarking setup. / Question: meaning of the commitment-related metrics.*

See A.3 (benchmark circuit, baselines) and A.5 (metric definitions, per-scheme commitment costs and their impact).

> *Question: how does the protocol description map to the gnark implementation, especially in the Fiat-Shamir setting?*

See A.4 — Section 6.3 traces Algorithms 4/5 call-by-call onto `MulPolyRings`, `polyRingMulHint`, and `performDeferredRingChecks`, whose two `api.Commit` calls realise the two Fiat-Shamir challenges of

Section 5.5.

Review 104C

> *Weakness: Table 6 — unclear what "Before" and "After" compare relative to this work, Fel23, Hou23, NE24.*

Section 7.2 now defines both columns: *Before* is Garaga verifying each extension-field multiplication individually via the polynomial identity testing of [Fel23]; *After* is Garaga using our consolidated relations. The relation of the technique to [Hou23] and [NE24] is covered in the rewritten Section 3.

> *Weakness: the "gnark baseline" in Table 7 should be precise about its state relative to recent academic results.*

Section 7.2 now states that the baseline is the upstream gnark `emulated` package including the optimizations of [Hou23] and [NE24] — i.e., the current state of the art for emulated pairings — and that our numbers come from the identical benchmark compiled against our fork, with no changes to the circuit definition or test harness.

> *Weakness: MULMOD, ADDMOD, COMMITS, VM STEPS never properly defined.*

Defined in the Table 6 footer and in the accompanying text of Section 7.2 (MULMOD/ADDMOD: modular field operations via built-in functions; COMMITS: Poseidon input commitments for Fiat-Shamir, cross-referenced to Section 5.5.2; VM STEPS: CairoVM CPU cycles).

> *Weakness: the toolkit is not evaluated beyond the pairing use case.*

We now position the toolkit explicitly as a reusable artefact of independent interest (Section 1.2) whose evaluation in this paper is via the pairing workload; Section 6 documents its generic interface (arbitrary modulus polynomial, multiple coexisting ring groups, accumulator with degree bounding), and the further-work discussion notes candidate applications beyond pairings. A dedicated multi-workload evaluation is beyond this paper's scope.

> *Editorial: the 29% / 60% / 42% points appear almost identically several times.*

Deduplicated — headline numbers (updated for the new uniform benchmark) now appear once in the abstract and once in the Section 1.2 contribution list; other occurrences were removed.

> *Editorial: broken sentence at 178–179.*

Fixed.

> *Editorial: 258–259 — consider not letting x be both a scalar and a formal intermediate.*

Adopted: Section 5.6 now opens by fixing the convention that z, $x \in F_q$ denote scalar challenges while polynomials use the formal indeterminate X (so z^j is a scalar coefficient), and the surrounding text was adjusted to respect it.

> *Editorial: the API description of 6.3–6.4 reads like library documentation and could move to an appendix.*

Restructured and condensed. We kept it in the main body because the binding criterion on algorithm-to-implementation mapping (A.4) is discharged there: the API reference is now followed by subsections that trace Algorithms 4/5 onto the implementation, rather than standing as reference documentation.

Part C — Location of changes in the revised paper

Location (revised)	Change	Criteria
Abstract	Rewritten results statement (both curves, uniform benchmark); terminology unified	A.2, A.3, A.5
Section 1.2	Contribution list rewritten; updated headline numbers; toolkit positioned as independent artefact	A.1, A.2, A.6
Section 3	Rewritten related work with per-work deltas; new Garaga paragraph	A.1
Section 4	Editorial fixes; sparseness accounting and k-pair degree scaling referenced from benchmarks	A.3, A.6
Section 5.4	Interactive protocol as explicit two-round message diagram; redundant floats removed	A.4, A.6
Section 5.5 (new)	Fiat-Shamir transformation: transcript, non-interactive proof, random-oracle instantiations with per-scheme commitment costs	A.4, A.5
Section 5.6	Notation convention (scalars vs. indeterminate); Fiat-Shamir soundness bound instantiated	A.4, A.6
Section 6.3 (new)	API reference plus explicit mapping of Algorithms 4/5 onto the gnark implementation	A.4
Section 7.2	Rewritten evaluation: Before/After and baseline definitions, new “Benchmark circuit” paragraph, metric definitions; Tables 6–7 regenerated with BLS12-381 and the uniform 3-pairing circuit	A.2, A.3, A.5

References follow the revised paper's bibliography: [Fel23] Feltroidprime, HackMD 2023; [Hou23] El Housni, CT-RSA 2023; [NE24] Novakovic–Eagen, ePrint 2024/640; [BSB23] Belling–Soleimanian–Bégassat, CCS 2023; [BSB] Belling et al., multi-round `api.Commit` proposal.